Software Testing

Tuesday, October 14, 2025 10:00 AM

Software Testing

Software testing is the process of **executing a program to identify errors** and verify that it performs as expected. The main goal is to ensure that the software **meets user requirements** and is **free from major defects** before deployment.
Testing is done using **artificial data** (test cases), and the results are checked for **errors, anomalies, or performance issues**.
Goals of Software Testing:

- To demonstrate to the developer and customer that the software **meets its specified requirements**.
- To **discover defects or incorrect behaviors** in the software.

Verification and Validation (V&V) Testing

Verification and **Validation** are two key activities in software quality assurance that ensure the final product is both **built correctly** and **is the right product** for the user.

Here's a clear, detailed explanation of the **difference between Verification and Validation** —

1. Meaning

- **Verification** checks *whether the software is being built correctly* according to design and requirements.
  It ensures the product **meets the specifications** set by developers or documents.
- **Validation** checks *whether the right software is being built* — that is, whether it **meets the actual needs** of users and customers.

2. Objective

- **Verification:** To ensure that the product is **technically correct** and follows the planned design and documentation.
- **Validation:** To ensure that the product is **practically useful** and solves the user's real problem.

3. When Performed

- **Verification:** Done **during development**, at every phase (requirements, design, coding).
- **Validation:** Done **after development**, when the complete system is available for testing.

4. Activities Involved

- **Verification:** Includes reviews, walkthroughs, and inspections — **no code execution**.
- **Validation:** Includes actual testing (unit, system, acceptance) — **involves code execution**.

5. Performed By

- **Verification:** Quality Assurance (QA) team or developers.
- **Validation:** Testing team or end-users.

Example
Imagine developing a **banking app**:

- **Verification:** Checking that the login module follows the design — e.g., password encryption and input validation work as documented.

- **Validation:** Checking that users can successfully log in and access their accounts without confusion — confirming it meets real-world user expectations.

In short:

- **Verification = Are we building the system right?**
- **Validation = Are we building the right system?**

Example:
In an **online shopping app**, developers check whether the "Add to Cart" button updates the database correctly — this is **verification** (making sure it's built right).
Later, users test whether they can easily search, add, and buy products — this is **validation** (making sure it's the right system for users).

Software Inspection

Software inspection is a **quality control technique** used to ensure that documentation or code produced during one phase of development is **consistent with earlier phases** and follows **defined rules and standards**.
It involves people **examining code, design, or documents** to find **errors, anomalies, or violations of standards** before testing begins.
Aim:
To **locate faults early** in development using a **fault checklist** rather than waiting for execution-based testing.

Roles in Inspection

Role	Description
Author	The person who created the document or code being inspected. They are responsible for correcting the defects found during the inspection. **Example:** A developer who wrote a login module and later fixes bugs identified in it.
Moderator	Leads and manages the inspection process — plans meetings, assigns roles, and ensures the discussion stays on track. **Example:** A team lead organizing the inspection session for a new payment feature.
Chief Moderator	Supervises the entire inspection process, maintains and updates the inspection checklists, and ensures overall quality improvement. **Example:** A senior QA manager who updates defect-check templates after each project.
Reader	Reads the document or code line by line during the meeting so the inspection team can follow and discuss it. **Example:** A tester reads the source code of a registration module aloud for the group to analyze.
Inspector	Examines the code or document carefully to detect errors, defects, or inconsistencies. **Example:** A quality analyst checking the logic of a discount calculation module to spot mistakes.

Inspection Process

1. **Planning:**
   The moderator plans the inspection, selects team members, and prepares materials.
2. **Overview:**
   The author presents the document or code to the inspection team for understanding.
3. **Individual Preparation:**
   Each team member independently reviews the material and notes down possible defects.

4. **Inspection Meeting:**
   The reader goes through the document part by part while inspectors identify and discuss defects.
5. **Rework:**
   The author corrects the identified issues according to the action plan.
6. **Follow-up:**
   The moderator checks if corrections are properly made. If necessary, a **re-inspection** may be done before final approval.

Example:
Suppose a team is developing an online banking app. Before testing the "Money Transfer" module, a **code inspection** is done to check logic consistency, variable naming, and error handling.
This helps detect potential issues early—saving time and cost compared to finding them during execution testing.

Software Testing Process

The software testing process begins with the preparation of test cases and proceeds through several stages to ensure that the software meets its requirements and functions correctly.

1. Preparation of Test Cases

Test cases specify:

- The inputs to be tested.
- The expected output from the system.
- The specific feature or function being tested.

These help ensure that every function of the software is validated.
Example:
If you are testing a login system:

- Input: Correct username and password
- Expected Output: Redirect to dashboard

From the design of test cases, we create a test case list.

2. Preparation of Test Data

Test data are the actual inputs designed to test the system.
They can be generated manually or automatically using testing tools.
Example:
In a banking app, test data may include:

- Valid account numbers
- Invalid numbers
- Negative or zero transaction amounts

3. Execution of Test

The program is run with test data for all test cases, and the actual results are recorded.

4. Comparison and Reporting

The actual results are compared with the expected results from the test cases.
A test report is generated showing:

- Passed test cases
- Failed test cases
- Defects found

Types / Levels of Software Testing

1. Unit Testing

Focuses on testing the smallest unit of software design (for example, a single function or module).

Usually done by programmers using sample input and checking the output.
Example:
Testing a function add(a,b) to ensure add(2,3) gives 5.

2. Integration Testing
After individual units are tested, they are combined and tested as a group.
Ensures that modules interact correctly.
Example:
Testing how the "Login" module integrates with the "User Dashboard" module.

3. System Testing
The complete and integrated software is tested as a whole.
Ensures that it meets all specified system requirements.
Example:
Running the entire e-commerce website to check user registration, cart, and payment systems together.

4. Acceptance Testing
Ensures the system meets customer requirements.
It is the final testing phase before release.
Types:
- **Alpha Testing:** Performed by internal QA team before public release to identify major bugs.
  Example: Developers test a new mobile app internally.
- **Beta Testing:** The product is released to a limited group of users in real conditions to collect feedback.
  Example: YouTube releasing a beta version of a new interface to selected users.

5. Regression Testing
Performed after modifications or new features are added.
Ensures that existing functionality still works correctly.
Example:
After adding a "Dark Mode" feature, testers recheck login, search, and settings functions.

Testing Methods
Software Testing Methods describe how testing is performed. There are two main approaches:

1. Black Box Testing
- **Definition:** Testing without knowing the internal workings of the software. The tester focuses on **inputs and expected outputs**.
- **Goal:** Verify that the system behaves as intended according to requirements.
- **Performed by:** Testers or end users.
- **Pros:**
  - Simple and quick.
  - Focuses on functional requirements.
- **Cons:**
  - Doesn't check internal code logic.
  - Limited in finding some types of errors.
- **Example:** Testing a login page by entering valid and invalid credentials to see if the system allows or blocks access.

2. White Box Testing
- **Definition:** Testing with knowledge of the **internal code, structure, or logic** of the software.
- **Goal:** Ensure all paths, conditions, and loops in the code work correctly.
- **Performed by:** Developers or testers with programming knowledge.
- **Pros:**

    - Thorough; can catch hidden errors in code.
    - Good for algorithm and logic testing.
- **Cons:**
    - Time-consuming.
    - Requires programming knowledge.
- **Example:** Checking if all branches of an if-else statement in the payment calculation module are executed correctly.

Black Box Testing	**White Box Testing**
Internal workings of the software are not known to the tester.	Tester has full knowledge of internal structure and code.
Based on requirement specification.	Based on detailed design or code.
Performed by testers or end users.	Performed by developers.
Suitable for functional testing.	Suitable for algorithm and logic testing.
Less time-consuming.	More exhaustive and time-consuming.

Example:

- Black Box: A tester inputs data in a login form without knowing the backend code.
- White Box: A developer checks the if-else conditions and loops in the login code.

Development Testing

Development testing is a method of applying testing practices consistently throughout the **software development life cycle (SDLC)**.
Its main goal is to **detect bugs and errors early**—during the development phase—so that time, cost, and effort are minimized later.
It helps establish a framework to verify whether the project requirements are being met and whether the system aligns with the intended mission or functionality.
This testing is **performed by software developers or engineers** during the **construction phase** of SDLC.

1. Test-Driven Development (TDD)

Test-Driven Development (TDD) is an approach where tests are written **before** the code is developed.
The process alternates between **writing a test**, **writing code**, and **refactoring** until the test passes.
Process:

1. Write a test for a small functionality.
2. Write the minimal code necessary to make the test pass.
3. Run all tests again to ensure nothing else is broken.
4. Refactor the code if necessary.
5. Repeat the cycle for the next feature.

Example:
Suppose you are developing a calculator app.

- First, you write a test for the function add(2,3) expecting output 5.
- Then you write the code for the add() function.
- Once the test passes, you move on to another function like subtract().

TDD ensures that every feature works correctly before proceeding to the next.

2. Release Testing

Release testing refers to the testing practices and strategies used to verify that a **software release candidate is ready for users**.
Its goal is to **find and eliminate remaining errors and bugs** so that the final version is stable and reliable.
Purpose:
To **convince the customer or management** that the software is good enough for use in real-world conditions.
Example:
Before launching a new version of a mobile banking app, the release team tests login, money transfer, and security features to ensure no major bugs exist in the production version.

3. User Testing

User testing (also called **Acceptance Testing by Users**) is the final stage of testing where **actual users or customers test the system** and provide feedback on its performance, usability, and reliability.
Even if system and release testing have been done thoroughly, user testing is essential because it exposes issues related to **real-world conditions** that may not appear in a controlled environment.
Example:
An educational app might work perfectly in a lab test but crash on older smartphones or slow internet connections.
User testing helps identify these real-world issues before public release.

Software Quality Assurance (SQA)

Software Quality Assurance (SQA) is a systematic process that ensures the **software development and maintenance processes** follow defined standards and procedures to achieve a high-quality product.
SQA works **parallel to software development**, focusing on **process improvement** rather than defect correction after development.
Key Focus:

- Ensuring adherence to software engineering standards.
- Preventing defects rather than detecting them late.
- Monitoring and improving development processes.

Example (Library Management System):

1. Create an **SQA Management Plan** defining how quality will be maintained throughout the project.
2. Set up **checkpoints and schedules** to track progress and deadlines.
3. Apply **software engineering techniques** such as design reviews, documentation checks, and testing at each stage.
4. Conduct **formal reviews** and audits to ensure each module meets the required quality standards.